

构造篇期末复习

依据：软件构造、代码设计课件，123.txt 答疑转写，考试题目类型与重点，往年卷构造题。

使用建议

构造题通常让你评价一段代码并改写。答题要按“正确性 - 可读性 - 可维护性 - 可靠性 - 测试/复杂度”的顺序分析，最后给出更清晰、更安全的伪代码或 Java 代码。

一、考试定位与优先级

优先级	内容	为什么重要	答题产物
高	代码质量评价：正确、易读、可靠、可维护	2020、2022 都有构造题要求分析代码优缺点	列问题 + 解释影响 + 给改写代码
高	命名、格式、注释、魔法数字、复杂判断	代码设计课件重点；往年卷代码故意写得很差	指出坏味道并重构
高	异常、断言、防御式编程、边界情况	答疑提到基本 assertion 概念要知道；构造题常考可靠性	输入校验、边界处理、异常语义
中高	圈复杂度	2022 测试题直接让算，但属于代码结构度量	按判定节点 + 1 或控制流图计算
中	重构、TDD、单元测试、代码评审、调试	构造活动常考简答或作为代码分析理由	定义、步骤、优点、适用时机
中	持续集成 CI 和工具链	2020 简答出现过	好处 + 典型工具链

一句话判断

构造题不是让你展示语法细节，而是让你证明自己能把代码写到“别人一看就明白、边界不出错、后续容易改”的程度。

二、软件构造基础

1. 什么是软件构造

软件构造是通过编码、验证、单元测试、集成测试和调试等工作，生产可工作的、有意义的软件的详细创建过程。它不是“只写代码”，而是设计落地、测试验证、调试修复、代码评审、构建集成和构造管理的组合。

软件构造活动

- 详细设计；
- 编程；
- 测试；
- 调试；
- 代码评审；
- 集成与构建（Build）；
- 构造管理。

课件截图：软件构造活动。构造包含详细设计调整、编程、测试、调试、评审、构建和管理。

2. 代码质量目标

质量	含义	构造题常见表现
易读性	代码显而易见是正确的，别人能快速理解	命名清晰、格式规范、控制结构简单、注释有意义
易维护性	代码容易修改和扩展	方法短小、职责单一、避免重复、避免魔法数字
可靠性	执行正确并能妥善处理故障	边界值、非法输入、异常、资源释放、状态一致性
性能	时间和空间效率合理	选择合适数据结构和算法，避免不必要重复计算
安全性	不泄露重要信息，不留下漏洞	输入校验、权限控制、避免越界和注入类问题

程序代码的典型质量

- 易读性。
 - 程序代码必须是易读的，看上去“显而易见是正确的”。易读性是编程最为重要的目标，它可以使得程序更容易开发，尤其是易于调试；可以使得程序更容易维护，减少理解代码的难度和成本；可以使得程序易于复用。
- 易维护性。
 - 除了易读之外，易维护性要求程序代码易于修改。
- 可靠性。
 - 程序代码必须是可靠的，要执行正确，并妥善处理故障。
- 性能。
 - 程序代码必须是高性能的，包括时间性能和空间性能，需要进行仔细的数据结构和算法设计。
- 安全性。
 - 不要遗留程序漏洞，不要出现重要信息的泄漏（例如内存数据区泄漏）。

课件截图：程序代码的典型质量。构造题可以按这五类逐条分析。

三、设计易读的代码

1. 格式和逻辑组织

- 使用缩进、对齐和空行表达逻辑结构。
- 相关逻辑放在一起，复杂控制结构之间用空行分割。
- 类定义内部可按成员变量、构造方法、`public`、`protected`、`private` 方法组织。
- 条件和循环即使只有一行，也建议使用 `{}`，减少维护时误解。

2. 命名

- 类、属性、数据用名词；方法用动词或动词+名词；接口用名词或形容词。
- 名称要表达真实职责，例如 `calculateStockDays` 比 `aa`、`dolt` 更好。
- 遵守语言命名惯例；避免易混字符、无意义缩写、过长名称。
- 临时变量可短，但业务变量必须有意义。

命名

- **使用有意义的名称进行命名。**例如对“销售信息”类，命名为 `Sales` 而不是 `ClassA`。
 - 使用名词命名类、属性和数据；
 - 使用名词或者形容词命名接口；
 - 使用动词或者“动词+名词”命名函数和方法；
 - 使用合适的命名将函数和方法定义的明显、清晰，包括返回值、参数、异常等。
- **名称要与实际内容相符。**例如，使用 `Payment` 命名“账单”类明显比使用“`Change`”更相符，因为“账单”类的职责不仅仅是计算“`Change`”，还要维护账单数据。
- **如果存在惯例，命名时要遵守惯例。**例如，Java 语言的命名惯例是：使用小写词命名包；类、接口名称中每个单词的首字母大写；变量、方法名称的第一个单词小写，后续单词的首字母大写；常量的每个单词大写，单词间使用“`_`”连接。

课件截图：命名规则。2020 构造题中的 `aa(int aa)` 是典型反例。

3. 注释

- 注释解释意图、约束、边界和复杂控制结构，不要重复代码字面含义。
- 方法文档应说明参数、返回值和可能异常。
- 注释必须和代码同步，错误注释比没有注释更危险。

内部注释

- 注释要有意义，不要简单重复代码的含义
- 重视对数据类型的注释
- 重视对复杂控制结构的注释

课件截图：内部注释。重点不是注释越多越好，而是解释代码没有直接表达的信息。

四、设计易维护的代码

1. 小型任务与复杂决策

- 每个函数/方法应内聚地完成一个目标；长方法说明它承担了太多任务。
- 复杂判断可提取为有意义的布尔变量或方法，例如 `isLeadTimeInThreeDays(days)`。
- 多分支规则可用 `else if`、表驱动、策略模式或规则对象，避免散乱的多独立 `if`。

18.2 设计易维护的代码

- 1 小型任务
 - 要让程序代码可修改，就要控制代码的复杂度。这首先要要求每个函数或方法的代码应该是内聚的，恰好完成一个功能与目标。
 - 如果内聚的代码本身比较简单，复杂性可控，那么它就具有比较好的可维护性。反之，内聚的代码也可以比较复杂，典型表现是完成一个功能需要多个步骤、代码比较长，那么就需要将其进一步分解为多个高内聚、低耦合的小型任务。

课件截图：小型任务。构造题中长方法、混合职责通常要拆分。

-
- 2 复杂决策
 - 使用新的布尔变量简化复杂决策
 - 使用有意义的名称封装复杂决策
 - 对于决策“`If (id>0) && (id<=MAX_ID)`”，可以封装为“`If (isIdValid(id))`”，方法`isIdValid(id)`的内容为“`return ((id>0) && (id<=MAX_ID))`”。
 - 表驱动编程

课件截图：复杂决策。把难懂条件封装成命名清晰的判断。

2. 数据使用

- 变量不要一变量多用，名称应与用途一致。
- 限制全局变量；必须使用时要说明含义和使用范围。
- 魔法数字/字符串要提取为具名常量，例如 `SEVEN_DAYS_STOCK = 7`。
- 方法参数不要过多；必要时用参数对象，但避免印记耦合。

数据使用

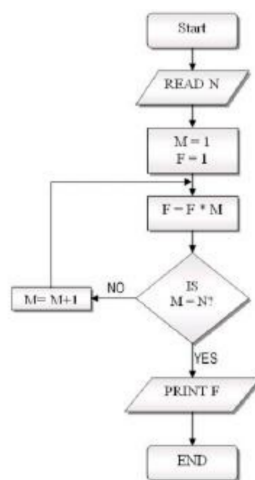
- (1) 不要将变量应用于与命名不相符的目的。例如使用变量 `total` 表示销售的总价，而不是临时客串 `for` 循环的计数器。
- (2) 不要将单个变量用于多个目的。在代码的前半部分使用 `total` 表示销售总价，在代码后半部分不再需要“销售总价”信息时再用 `total` 客串 `for` 循环的计数器也是不允许的。
- (3) 限制全局变量的使用，如果不得不使用全局变量，就明确注释全局变量的声明和使用处。
- (4) 不要使用突兀的数字与字符，例如 15（天）、“MALE”等，要将它们定义为常量或变量后使用。

课件截图：数据使用。魔法数字、变量复用和全局变量是构造题常见扣分点。

五、设计可靠的代码

1. 异常、断言、防御式编程

方式	适用	考试表述
异常	处理运行时可能发生、调用者需要知道的错误	非法参数、外部资源失败、业务前置条件不满足时抛出或返回明确错误
断言	检查程序员假设/内部不变式，主要用于调试和契约检查	基本概念要知道，不考特殊语法；可用伪代码表示 assert
防御式编程	不能保证外部输入和环境正确时，保护内部状态	检查输入、边界、空值、资源状态和外部方法返回值



程序流程图

课件截图：可靠代码。异常、断言、防御式编程分别服务于不同错误场景。

2. 常见错误与修复

- 边界错误：`<` 和 `<=` 混用，导致边界值走错分支。
- 条件遗漏：规则区间不连续，某些输入无法得到合理结果。
- 逻辑错误：多个独立 `if` 可能相互覆盖，应使用互斥 `else if`。
- 硬编码：规则数字散落在代码中，变更时容易漏改。
- 异常缺失：负数、空值、未知类型未处理。

修复缺陷的注意点

- 一次只修复一个缺陷。
- 修改前保留旧版本的备份，如果项目使用了配置管理系统，这个工作会由配置管理工具完成，否则就需要由程序员手动完成。
- 使用测试和评审验证修复的有效性。
- 检查和修复类似的缺陷，这可以在代码搜索、程序切片等工具的帮助下进行。

课件截图：修复缺陷注意点。一次修复一个缺陷，并用测试和评审验证。

六、重构、评审、调试与 TDD

1. 重构

重构是在不改变外部行为的前提下改进内部结构。它不能用来实现新功能，而是在新增功能后、修复缺陷时、代码评审发现坏味道时，用来降低复杂度和改善可维护性。

坏味道	问题	常见重构
长方法	职责太多、难测试	提取方法、拆分步骤
过多参数	接口不简洁，调用方负担重	拆方法或引入参数对象
重复代码	修改易漏	提取公共方法
魔法数字/字符串	含义不清，规则变更难	具名常量、枚举、配置表
深层嵌套	圈复杂度、高、难读	卫语句、提取条件、策略模式

重构

- 为什么要重构？
 - 1、因为无法预计到后续数年的修改，导致软件开发阶段的设计方案不能满足修改要求；
 - 2、随着修改次数的增多，软件设计结构的质量越来越脆弱，很难继续维持可修改性。
- 什么是重构？
 - 修改软件系统的严谨方法，它在不改变代码外部表现的情况下改进其内部结构。
- 重构的时机？
 - 增加新的功能时。需要注意的是重构发生在新功能增加完成之后，用来消除新功能所添加代码导致的坏味道；而不是发生在新功能添加之前，重构不改变代码外部行为，不是能够实现新功能添加的方法。
 - 发现了缺陷进行修复时。诊断缺陷时如果发现代码存在坏味道或者修复代码会引入坏味道，就需要进行重构。
 - 进行代码评审时。如果在评审代码时发现了坏味道，就需要进行重构。

课件截图：重构。构造题给改写代码时，本质就是一次小型重构。

2. 代码评审

- 代码评审是对代码的系统检查，可发现代码错误、坏味道和规范问题。
- 形式包括正式评审、轻量级评审和结对编程。
- 评审点：逻辑正确性、边界处理、异常处理、命名格式、重复代码、接口依赖、安全风险。

代码评审

- 代码评审对代码的系统检查，通常是通过同行专家评审来完成的。通过评审会议可以发现并修正之前忽略的代码错误，从而同时提高软件的质量和开发者的技巧。
- 代码评审一般分为正式评审、轻量级评审和结对编程。

课件截图：代码评审。AI 可以辅助发现坏味道，但最终判断仍需要开发者理解业务。

3. 调试

- 调试目标是定位并修复缺陷，过程包括重现问题、诊断缺陷、修复缺陷。
- 重现问题可控制输入或控制环境；定位时先检查刚修改过的部分。
- 修复后要补测试，检查同类缺陷是否还存在。

4. TDD

- TDD 循环：Red 先写失败测试，Green 写最少代码通过测试，Refactor 在测试保护下重构。
- 优点：驱动接口设计、提高可测试性、保护重构、减少回归缺陷。
- 注意：测试用例仍要由开发者审查业务正确性，不能盲信自动生成测试。

```

public class GetChangeOfSalesTester {
    public static void main(String[] args){
        Sales sale=new Sales();
        // 购买 2 个 ID=1 的商品, 该商品的测试数据 Price=45
        sale.addSalesLineItem(1, 2)
        sale.total();
        double change;
        boolean passed=true;
        change= sale.getChange(-100);    //测试用例 1
        if (change!= -1) {
            passed=false;
            System.out.println("parameter exception not handled");
        }
        change= sale.getChange(0);    //测试用例 2
        if (change!= 1) {
            passed=false;
            System.out.println("parameter exception not handled");
        }
        change= sale.getChange(50);    //测试用例 3
        if (change!= 2) {
            passed=false;
            System.out.println("parameter&&attribute exception not handled");
        }
        change= sale.getChange(90);    //测试用例 4
        if (change!= 0) {
            passed=false;
            System.out.println("when parameter=attribute, error");
        }
        change= sale.getChange(100);    //测试用例 5
        if (change!= 10) {
            passed=false;
            System.out.println("when parameter>attribute, error");
        }
        if (passed==true)    //所有测试用例通过
            System.out.println("None error found");
    }
}

```

课件截图：测试驱动开发。构造阶段单元测试和重构互相支撑。

七、圈复杂度

1. 计算方法

- 控制流图公式：`V(G) = E - N + 2P`，一般单个方法 P=1。
- 考试快速算法：圈复杂度 = 判定节点数 + 1。
- 常见判定节点：`if`、`else if`、`while`、`for`、`case`、`catch`、三目运算等。
- 复合条件是否额外计数要看课程/题目要求。若按基础 McCabe，通常每个判定语句 +1；若题目考条件覆盖，短路条件要单独分析测试。

18.6 代码复杂度度量

- 程序复杂度是造成各种编程困难的主要原因。
[Dijkstra1972]很早就指出：“有能力的程序员会充分认识到自己的大脑容量是多么地有限；所以，他会非常谦卑的处理编程任务。”
- 为了帮助程序员处理程序复杂度，人们提出了很多程序复杂度的度量手段，其中McCabe的圈复杂度[McCabe1976]得到了比较大的关注。

课件截图：圈复杂度。用它判断代码是否过复杂、是否需要重构。

2. 常见例子

```
int f(int x) {
    if (x < 0) return -1;    // +1
    if (x == 0) return 0;   // +1
    return 1;
}
// ■■■■ = 2 ■ if + 1 = 3

void g(int x) {
    switch (x) {
        case 1: ...
        case 2: ...
        default: ...
    }
}
// ■■■■■■ case ■■■■■■■■■■■■■■■■■■■■
```

八、往年卷风格例题与答案

例题 1：2020 订货规则代码评价与改写

原题代码大意：根据供应商交货期返回备货销量天数，但代码写成 `public int aa(int aa){...}`，使用多个独立 if 和魔法数字。

```
public int aa(int aa){int bb = 0;if(aa < 3) bb= 7;
if(aa < 7 && aa >=3) bb = 15;
if(aa < 10 && aa >= 7 ) bb = 20;
if(aa < 15 && aa >= 10) bb = 30;
if(aa >= 20) bb = 40;
return bb;}
```

1. 可指出的问题

- 命名差：`aa`、`bb` 完全不能表达含义，违反易读性要求。
- 格式差：所有语句挤在一起，缺少缩进、换行和代码块。
- 魔法数字：3、7、10、15、20、40 等规则数字散落，缺少常量名。
- 边界错误：题目说“3 天以内/7 天以内”，通常应包含边界，代码用 `<` 排除了 3、7、10、15。
- 规则区间不连续：15 到 19 天没有分支，会返回默认 0，可靠性差。
- 多个独立 if：规则本应互斥，应该使用 `else if` 或表驱动表达。
- 缺少非法输入处理：负数交货期不应正常返回。

2. 改写示例

```
private static final int STOCK_DAYS_FOR_3_DAYS = 7;
private static final int STOCK_DAYS_FOR_7_DAYS = 15;
private static final int STOCK_DAYS_FOR_10_DAYS = 20;
private static final int STOCK_DAYS_FOR_15_DAYS = 30;
private static final int STOCK_DAYS_FOR_20_OR_MORE_DAYS = 40;

public int calculateStockDays(int leadTimeDays) {
    if (leadTimeDays < 0) {
        throw new IllegalArgumentException("leadTimeDays must be non-negative");
    }
    if (leadTimeDays <= 3) {
        return STOCK_DAYS_FOR_3_DAYS;
    } else if (leadTimeDays <= 7) {
        return STOCK_DAYS_FOR_7_DAYS;
    } else if (leadTimeDays <= 10) {
        return STOCK_DAYS_FOR_10_DAYS;
    } else if (leadTimeDays <= 15) {
        return STOCK_DAYS_FOR_15_DAYS;
    } else if (leadTimeDays >= 20) {
        return STOCK_DAYS_FOR_20_OR_MORE_DAYS;
    }
    throw new IllegalArgumentException("no reorder rule for leadTimeDays");
}
```

关于 16-19 天

原题规则写成“15 天以内”和“20 天以上”，中间 16-19 天语义不明确。可靠代码不能悄悄返回 0；应抛异常或在答案中说明需要澄清需求。如果老师默认规则连续，也可把最后一段写成 `else return 40`。

例题 2：2022 发券代码优缺点

```
void issueCoupon(List<User> users){
    for(user:users){
        addCoupon(user,user.getType());
    }
}
void addCoupon(User u,int type){
    switch(type){
        case STUDENT:u.addCoupon(new FiveYuanCoupon());break;
        case TEACHER:u.addCoupon(new TenYuanCoupon());break;
        ...
    }
}
```

1. 可说的优点

- 结构直接，容易看出遍历用户并按类型发券。
- 发券逻辑集中在`addCoupon`中，比散落在多处略好。

2. 主要问题

- 控制耦合：`type`作为 flag 控制`addCoupon`内部 switch，调用者和被调用者都要知道类型含义。
- 违反 OCP：新增用户类型或优惠券类型要修改`switch`。
- 违反 DIP：方法直接依赖具体优惠券类`FiveYuanCoupon`、`TenYuanCoupon`。
- 缺少防御式编程：`users`、`user`可能为空，未知 type 没有处理。
- 命名和格式可继续改善，例如泛型、变量名、空格、default 分支。

3. 改写方向

```

interface CouponPolicy {
    Coupon createCouponFor(User user);
}

class StudentCouponPolicy implements CouponPolicy {
    public Coupon createCouponFor(User user) {
        return new FiveYuanCoupon();
    }
}

class TeacherCouponPolicy implements CouponPolicy {
    public Coupon createCouponFor(User user) {
        return new TenYuanCoupon();
    }
}

class CouponIssuer {
    private final Map<UserType, CouponPolicy> policies;

    public void issueCoupons(List<User> users) {
        if (users == null) {
            throw new IllegalArgumentException("users must not be null");
        }
        for (User user : users) {
            issueCoupon(user);
        }
    }

    private void issueCoupon(User user) {
        if (user == null) {
            return;
        }
        CouponPolicy policy = policies.get(user.getType());
        if (policy == null) {
            throw new IllegalArgumentException("unsupported user type");
        }
        user.addCoupon(policy.createCouponFor(user));
    }
}

```

- 这相当于用策略/工厂思想封装发券变化点。
- 新增类型时新增 `CouponPolicy`，尽量不改原有流程，符合 OCP。
- 业务流程依赖 `CouponPolicy` 抽象，符合 DIP。

九、构造题答题模板

1. 分析代码问题模板

- 正确性：边界是否对、分支是否完整、默认返回是否合理。
- 可读性：命名、缩进、空行、注释、括号、逻辑组织。
- 可维护性：魔法数字、重复代码、长方法、复杂判断、控制耦合。
- 可靠性：非法输入、空值、未知类型、异常、资源释放。
- 可测试性：分支是否容易覆盖，是否可用 Mock 隔离依赖。

2. 改代码模板

1. Rename: use meaningful method/variable names.
2. Validate: check invalid input early.
3. Constants: replace magic numbers with named constants.
4. Branches: use clear if/else-if or table-driven rules.
5. Refactor: extract complex decisions into named methods.
6. Test: cover normal, boundary, and abnormal cases.

3. 简答题速记

概念	速记答案
软件构造	通过编码、验证、单元测试、集成测试、调试等工作生产可工作软件的过程
调试	重现问题、诊断缺陷、修复缺陷
重构	不改变外部行为，改善内部结构
TDD	Red-Green-Refactor
代码评审	同行系统检查代码，发现缺陷和坏味道
CI	频繁集成、自动构建、自动测试，尽早发现集成问题
圈复杂度	控制流图复杂度；快速算法为判定节点数 + 1

4. 持续集成 CI

- 好处：尽早发现集成错误，减少“大爆炸式集成”风险；自动运行构建和测试，提高反馈速度；保证主干代码持续可工作。
- 工具链例子：Git/GitHub/GitLab 管理版本；Maven/Gradle/npm 构建；JUnit/pytest 运行测试；GitHub Actions/GitLab CI/Jenkins 执行流水线；SonarQube 做静态分析。